

LAB EXPERIMENTS: TOPIC 3

TOPIC 3 – DIVIDE AND CONQUER

1. Maximum and Minimum Using Divide and Conquer

Aim

To find the maximum and minimum elements in an array using the Divide and Conquer technique.

Algorithm

1. Divide the array into two halves.
2. Recursively find the minimum and maximum of both halves.
3. Compare the two minimum values and two maximum values.
4. Return the final minimum and maximum.

Pseudocode

```
MaxMin(arr, low, high)
    if low == high
        return arr[low], arr[low]

    mid = (low + high) / 2
    min1, max1 = MaxMin(arr, low, mid)
    min2, max2 = MaxMin(arr, mid+1, high)

    minimum = min(min1, min2)
    maximum = max(max1, max2)

    return minimum, maximum
```

Time Complexity

$O(n)$

Space Complexity

$O(\log n)$

Code

```
def max_min(arr, low, high):
    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2
    min1, max1 = max_min(arr, low, mid)
    min2, max2 = max_min(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)

arr = [5, 7, 3, 4, 9, 12, 6, 2]
mn, mx = max_min(arr, 0, len(arr) - 1)

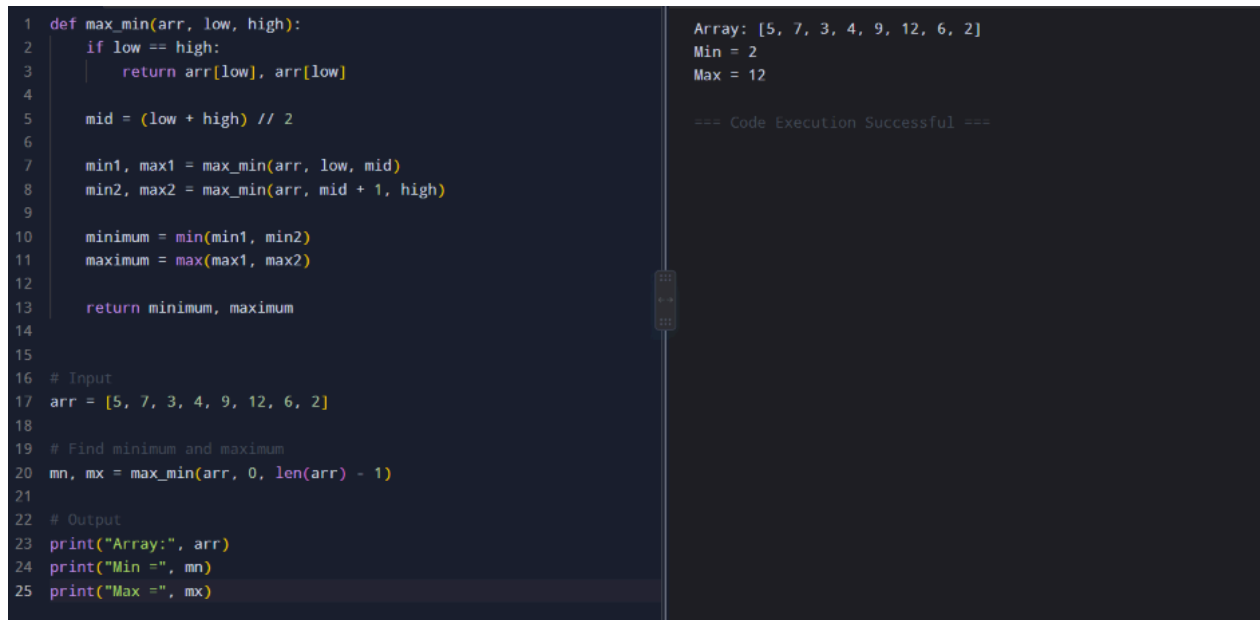
print("Min =", mn)
print("Max =", mx)
```

Input

N = 8
5, 7, 3, 4, 9, 12, 6, 2

Output

Min = 2
Max = 12



```
1 def max_min(arr, low, high):
2     if low == high:
3         return arr[low], arr[low]
4
5     mid = (low + high) // 2
6
7     min1, max1 = max_min(arr, low, mid)
8     min2, max2 = max_min(arr, mid + 1, high)
9
10    minimum = min(min1, min2)
11    maximum = max(max1, max2)
12
13    return minimum, maximum
14
15
16 # Input
17 arr = [5, 7, 3, 4, 9, 12, 6, 2]
18
19 # Find minimum and maximum
20 mn, mx = max_min(arr, 0, len(arr) - 1)
21
22 # Output
23 print("Array:", arr)
24 print("Min =", mn)
25 print("Max =", mx)
```

Array: [5, 7, 3, 4, 9, 12, 6, 2]
Min = 2
Max = 12

=== Code Execution Successful ===

Result

Thus, the minimum value is **2** and the maximum value is **12**.

2. Maximum and Minimum in a Sorted Array

Aim

To find the minimum and maximum values in a sorted array using Divide and Conquer.

Algorithm

1. Divide the array into two halves.
2. Recursively find minimum and maximum.
3. Compare the results from both halves.
4. Display the minimum and maximum.

Pseudocode

```
MaxMin(arr, low, high)
    if low == high
        return arr[low], arr[low]

    mid = (low + high) / 2

    min1, max1 = MaxMin(arr, low, mid)
    min2, max2 = MaxMin(arr, mid+1, high)

    return min(min1,min2), max(max1,max2)
```

Time Complexity

$O(n)$

Space Complexity

$O(\log n)$

Code

```
def max_min(arr, low, high):  
    if low == high:  
        return arr[low], arr[low]  
  
    mid = (low + high) // 2  
  
    min1, max1 = max_min(arr, low, mid)  
    min2, max2 = max_min(arr, mid + 1, high)  
  
    return min(min1, min2), max(max1, max2)
```

```
arr = [2, 4, 6, 8, 10, 12, 14, 18]  
mn, mx = max_min(arr, 0, len(arr) - 1)
```

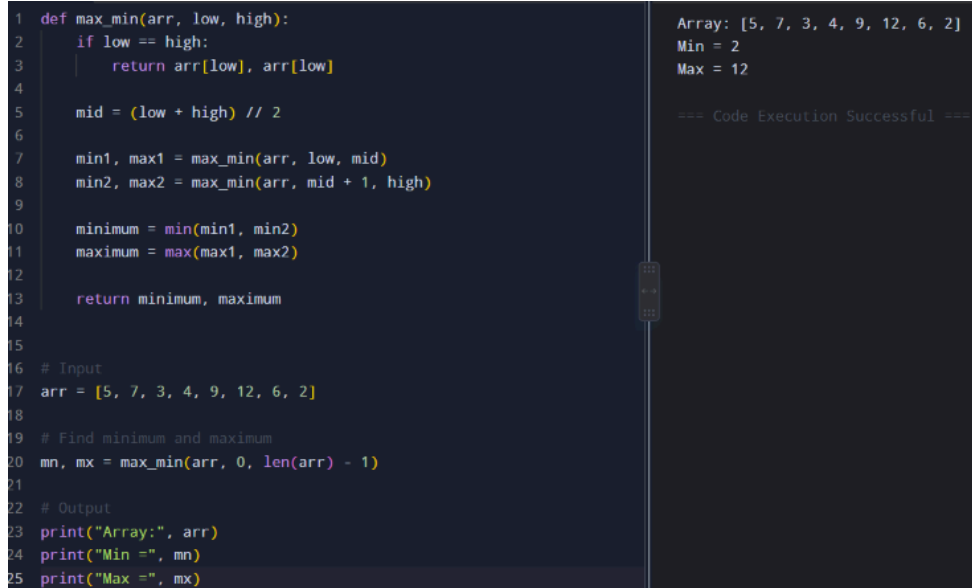
```
print("Min =", mn)  
print("Max =", mx)
```

Input

```
N = 8  
2, 4, 6, 8, 10, 12, 14, 18
```

Output

```
Min = 2  
Max = 18
```



```
1 def max_min(arr, low, high):  
2     if low == high:  
3         return arr[low], arr[low]  
4  
5     mid = (low + high) // 2  
6  
7     min1, max1 = max_min(arr, low, mid)  
8     min2, max2 = max_min(arr, mid + 1, high)  
9  
10    minimum = min(min1, min2)  
11    maximum = max(max1, max2)  
12  
13    return minimum, maximum  
14  
15  
16 # Input  
17 arr = [5, 7, 3, 4, 9, 12, 6, 2]  
18  
19 # Find minimum and maximum  
20 mn, mx = max_min(arr, 0, len(arr) - 1)  
21  
22 # Output  
23 print("Array:", arr)  
24 print("Min =", mn)  
25 print("Max =", mx)
```

Array: [5, 7, 3, 4, 9, 12, 6, 2]
Min = 2
Max = 12
--- Code Execution Successful ---

Result

The minimum and maximum values were successfully found as **2 and 18**.

3. Merge Sort

Aim

To implement Merge Sort for sorting an unsorted array using Divide and Conquer.

Algorithm

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves.
4. Continue until the complete array is sorted.

Pseudocode

```
MergeSort(arr)
  if size of arr > 1
    divide arr into left and right
    MergeSort(left)
    MergeSort(right)
    Merge left and right in sorted order
```

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
```

```
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])

result = []
i = j = 0

while i < len(left) and j < len(right):
    if left[i] < right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

result.extend(left[i:])
result.extend(right[j:])

return result
```

```
arr = [31, 23, 35, 27, 11, 21, 15, 28]
print("Sorted array:", merge_sort(arr))
```

Input

31, 23, 35, 27, 11, 21, 15, 28

Output

11, 15, 21, 23, 27, 28, 31, 35

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4
5     mid = len(arr) // 2
6
7     left = merge_sort(arr[:mid])
8     right = merge_sort(arr[mid:])
9
10    result = []
11    i = 0
12    j = 0
13
14    while i < len(left) and j < len(right):
15        if left[i] < right[j]:
16            result.append(left[i])
17            i += 1
18        else:
19            result.append(right[j])
20            j += 1
21
22    result.extend(left[i:])
23    result.extend(right[j:])
24
25    return result
26
27
```

Original array: [31, 23, 35, 27, 11, 21, 15, 28]
Sorted array: [11, 15, 21, 23, 27, 28, 31, 35]

=== Code Execution Successful ===

Result

The given array was successfully sorted using Merge Sort.

4. Merge Sort with Comparison Count

Aim

To implement Merge Sort and count the number of comparisons made during sorting.

Algorithm

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Count every comparison during merging.
4. Merge the two sorted halves.
5. Display the sorted array and comparison count.

Pseudocode

MergeSort(arr)

if size <= 1
return arr

divide array
sort left half

sort right half
merge both halves
increase count for every comparison

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Code

```
count = 0

def merge_sort(arr):
    global count

    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        count += 1

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```



```
arr = [12, 4, 78, 23, 45, 67, 89, 1]
sorted_arr = merge_sort(arr)
```

```
print("Sorted array:", sorted_arr)
print("Comparisons:", count)
```

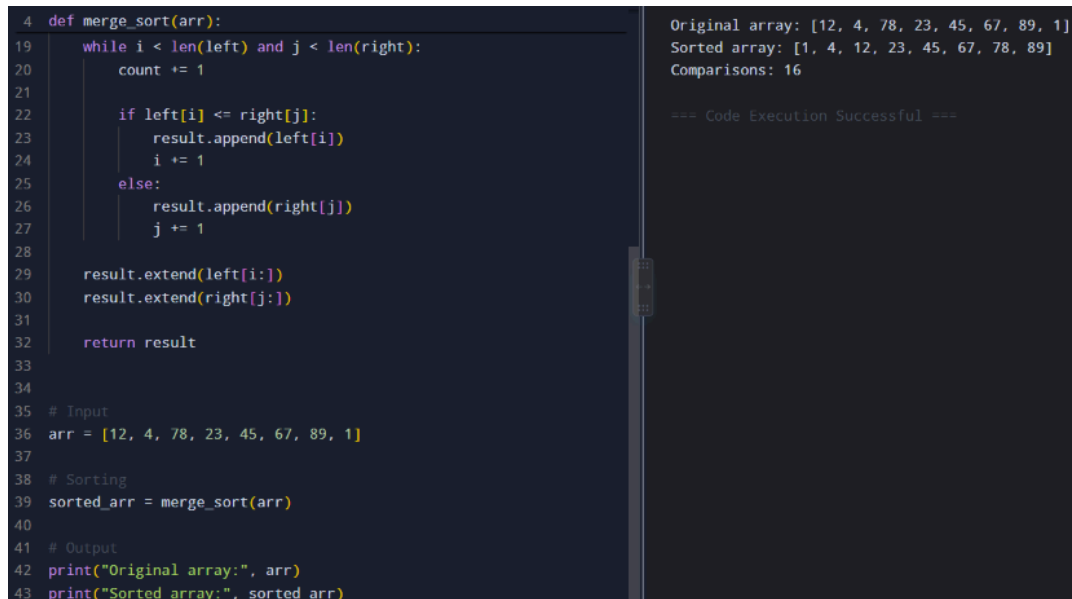
Input

12, 4, 78, 23, 45, 67, 89, 1

Output

Sorted array: [1, 4, 12, 23, 45, 67, 78, 89]

Comparisons: 16



```
4 def merge_sort(arr):
19     while i < len(left) and j < len(right):
20         count += 1
21
22         if left[i] <= right[j]:
23             result.append(left[i])
24             i += 1
25         else:
26             result.append(right[j])
27             j += 1
28
29     result.extend(left[i:])
30     result.extend(right[j:])
31
32     return result
33
34
35 # Input
36 arr = [12, 4, 78, 23, 45, 67, 89, 1]
37
38 # Sorting
39 sorted_arr = merge_sort(arr)
40
41 # Output
42 print("Original array:", arr)
43 print("Sorted array:", sorted_arr)
```

Original array: [12, 4, 78, 23, 45, 67, 89, 1]
Sorted array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

=== Code Execution Successful ===

Result

The array was successfully sorted and the number of comparisons was counted.

5. Quick Sort Using First Element as Pivot

Aim

To implement Quick Sort by choosing the first element as the pivot.

Algorithm

1. Select the first element as pivot.
2. Place smaller elements before the pivot.
3. Place larger elements after the pivot.
4. Recursively apply Quick Sort to both sub-arrays.
5. Display the sorted array.

Pseudocode

```
QuickSort(arr)
  if low < high
    pivot = first element
    partition array
    QuickSort(left part)
    QuickSort(right part)
```

Time Complexity

Average: $O(n \log n)$

Worst: $O(n^2)$

Space Complexity

Average: $O(\log n)$

Code

```
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[low]
        i = low + 1
        j = high

        while i <= j:
            while i <= high and arr[i] <= pivot:
                i += 1
            while arr[j] > pivot:
                j -= 1

            if i < j:
                arr[i], arr[j] = arr[j], arr[i]

        arr[low], arr[j] = arr[j], arr[low]
```

```
print("After partition:", arr)
```

```
quick_sort(arr, low, j - 1)  
quick_sort(arr, j + 1, high)
```

```
arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]  
quick_sort(arr, 0, len(arr) - 1)
```

```
print("Sorted array:", arr)
```

Input

10, 16, 8, 12, 15, 6, 3, 9, 5

Output

Sorted array: [3, 5, 6, 8, 9, 10, 12, 15, 16]

```
1 def quick_sort(arr, low, high):  
2     if low < high:  
8         while i <= j:  
10            while i <= high and arr[i] <= pivot:  
12  
13                while arr[j] > pivot:  
14                    j -= 1  
15  
16                if i < j:  
17                    arr[i], arr[j] = arr[j], arr[i]  
18  
19            arr[low], arr[j] = arr[j], arr[low]  
20  
21            print("After partition:", arr)  
22  
23            quick_sort(arr, low, j - 1)  
24            quick_sort(arr, j + 1, high)  
25  
26  
27 # Input  
28 arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]  
29  
30 # Sorting  
31 quick_sort(arr, 0, len(arr) - 1)  
32  
33 # Output
```

```
After partition: [6, 5, 8, 9, 3, 10, 15, 12, 16]  
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]  
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]  
After partition: [3, 5, 6, 8, 9, 10, 15, 12, 16]  
After partition: [3, 5, 6, 8, 9, 10, 12, 15, 16]  
Sorted array: [3, 5, 6, 8, 9, 10, 12, 15, 16]  
  
=== Code Execution Successful ===
```

Result

The array was successfully sorted using Quick Sort with the first element as pivot.

6. Quick Sort Using Middle Element as

Pivot

Aim

To implement Quick Sort using the middle element as the pivot.

Algorithm

1. Select the middle element as pivot.
2. Partition the array around the pivot.
3. Recursively sort the left and right parts.
4. Display the array after each partition.
5. Print the final sorted array.

Pseudocode

```
QuickSort(arr, low, high)
    choose middle element as pivot
    partition elements around pivot
    QuickSort(left)
    QuickSort(right)
```

Time Complexity

Average: $O(n \log n)$

Worst: $O(n^2)$

Space Complexity

$O(\log n)$ average

Code

```
def quick_sort(arr, low, high):
    if low >= high:
        return

    i, j = low, high
    pivot = arr[(low + high) // 2]

    while i <= j:
        while arr[i] < pivot:
            i += 1
```

```

while arr[j] > pivot:
    j -= 1

if i <= j:
    arr[i], arr[j] = arr[j], arr[i]
    i += 1
    j -= 1

print("After partition:", arr)

if low < j:
    quick_sort(arr, low, j)
if i < high:
    quick_sort(arr, i, high)

arr = [19, 72, 35, 46, 58, 91, 22, 31]
quick_sort(arr, 0, len(arr) - 1)

print("Sorted array:", arr)

```

Input

19, 72, 35, 46, 58, 91, 22, 31

Output

Sorted array: [19, 22, 31, 35, 46, 58, 72, 91]

```

1 def quick_sort(arr, low, high):
2     while i <= j:
3         while arr[j] > pivot:
4             j -= 1
5
6         if i <= j:
7             arr[i], arr[j] = arr[j], arr[i]
8             i += 1
9             j -= 1
10
11     print("After partition:", arr)
12
13     if low < j:
14         quick_sort(arr, low, j)
15
16     if i < high:
17         quick_sort(arr, i, high)
18
19 # Input
20 arr = [19, 72, 35, 46, 58, 91, 22, 31]
21
22 # Sorting
23 quick_sort(arr, 0, len(arr) - 1)
24
25 # Output

```

After partition: [19, 31, 35, 22, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 72, 46, 91]
After partition: [19, 22, 31, 35, 58, 46, 72, 91]
After partition: [19, 22, 31, 35, 46, 58, 72, 91]
Sorted array: [19, 22, 31, 35, 46, 58, 72, 91]

=== Code Execution Successful ===

Result

The array was successfully sorted using the middle element as pivot.

7. Binary Search with Comparison Count

Aim

To implement Binary Search and find the position of a given element while counting comparisons.

Algorithm

1. Find the middle element.
2. Compare it with the search key.
3. If equal, return its position.
4. If the key is smaller, search the left half.
5. Otherwise search the right half.
6. Count the comparisons.

Pseudocode

```
BinarySearch(arr, key)
    low = 0
    high = n-1
    while low <= high
        mid = (low + high) / 2
        count comparison

        if arr[mid] == key
            return mid
        else if key < arr[mid]
            high = mid - 1
        else
            low = mid + 1

    return -1
```

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$

Code

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if arr[mid] == key:
            return mid, comparisons
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1, comparisons

arr = [5, 10, 15, 20, 25, 30, 35, 40, 45]
index, comparisons = binary_search(arr, 20)
print("Position =", index + 1)
print("Comparisons =", comparisons)
```

Input

5,10,15,20,25,30,35,40,45
Search key = 20

Output

Position = 4

```
1 def binary_search(arr, key):
2     low = 0
3     high = len(arr) - 1
4     comparisons = 0
5
6     while low <= high:
7
8         mid = (low + high) // 2
9         comparisons += 1
10
11         if arr[mid] == key:
12             return mid, comparisons
13
14         elif key < arr[mid]:
15             high = mid - 1
16
17         else:
18             low = mid + 1
19
20     return -1, comparisons
21
22
23 # Input
24 arr = [5, 10, 15, 20, 25, 30, 35, 40, 45]
25 key = 20
26
```

```
After partition: [19, 31, 35, 22, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 72, 46, 91]
After partition: [19, 22, 31, 35, 58, 46, 72, 91]
After partition: [19, 22, 31, 35, 46, 58, 72, 91]
Sorted array: [19, 22, 31, 35, 46, 58, 72, 91]

=== Code Execution Successful ===
```

Result

The element **20** was found successfully at position **4**.

8. Binary Search – Mid-Point Calculation

Aim

To find an element in a sorted array using Binary Search and show the mid-point calculations.

Algorithm

1. Set low = 0 and high = n-1.
2. Calculate mid.
3. Compare the middle element with the key.
4. Reduce the search range.
5. Repeat until the key is found.

Pseudocode

low = 0

high = n-1


```
while low <= high
    mid = (low + high) / 2
    if arr[mid] == key
        return mid
    else if key < arr[mid]
        high = mid - 1
    else
        low = mid + 1
```

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$

Input

Array = 3, 9, 14, 19, 25, 31, 42, 47, 53
Key = 31

Mid-point Steps

low = 0, high = 8
mid = $(0+8)/2 = 4$
arr[4] = 25

31 > 25
Search right half.

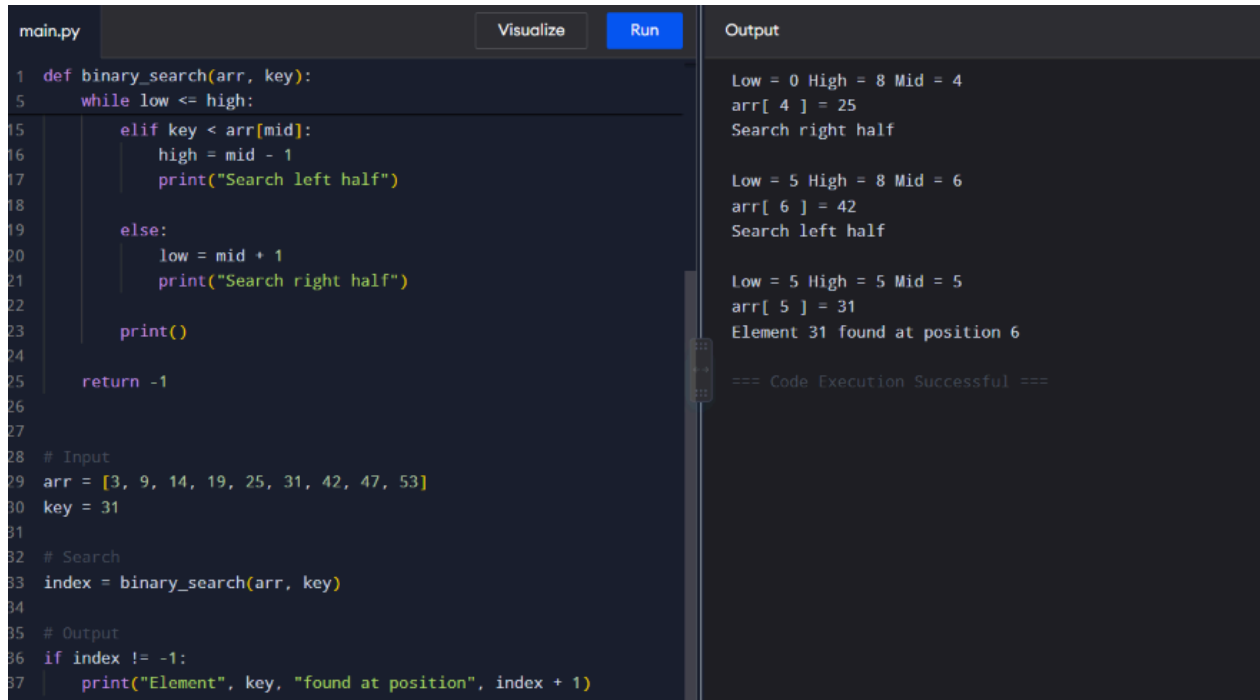
low = 5, high = 8
mid = $(5+8)/2 = 6$
arr[6] = 42

31 < 42
Search left half.

low = 5, high = 5
mid = $(5+5)/2 = 5$
arr[5] = 31

Output

Element 31 found at position 6



```
main.py Visualize Run Output
1 def binary_search(arr, key):
5     while low <= high:
15         elif key < arr[mid]:
16             high = mid - 1
17             print("Search left half")
18
19         else:
20             low = mid + 1
21             print("Search right half")
22
23         print()
24
25     return -1
26
27
28 # Input
29 arr = [3, 9, 14, 19, 25, 31, 42, 47, 53]
30 key = 31
31
32 # Search
33 index = binary_search(arr, key)
34
35 # Output
36 if index != -1:
37     print("Element", key, "found at position", index + 1)

Low = 0 High = 8 Mid = 4
arr[ 4 ] = 25
Search right half

Low = 5 High = 8 Mid = 6
arr[ 6 ] = 42
Search left half

Low = 5 High = 5 Mid = 5
arr[ 5 ] = 31
Element 31 found at position 6

=== Code Execution Successful ===
```

If the Array is Not Sorted

Binary Search depends on the array being sorted. If the array is not sorted, the algorithm may eliminate the wrong half and give an incorrect result. Therefore, an unsorted array should be sorted before applying Binary Search.

Result

The element **31** was successfully found at position **6**.

9. K Closest Points to Origin

Aim

To find the K closest points to the origin using a Divide and Conquer based sorting approach.

Algorithm

1. Calculate the squared distance of every point from the origin.
2. Sort the points according to their distance.

3. Select the first K points.
4. Display them.

Pseudocode

For each point:

distance = $x*x + y*y$

Sort points according to distance

Return first k points

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

Code

```
def k_closest(points, k):  
    points.sort(key=lambda p: p[0]**2 + p[1]**2)  
    return points[:k]
```

```
points = [[1, 3], [-2, 2], [5, 8], [0, 1]]  
k = 2  
print(k_closest(points, k))
```

Input

```
Points = [[1,3],[-2,2],[5,8],[0,1]]  
K = 2
```

Output

```
[-2, 2], [0, 1]
```

```
1 def k_closest(points, k):
2     points.sort(key=lambda p: p[0] ** 2 + p[1] ** 2)
3     return points[:k]
4
5
6 # Input
7 points = [[1, 3], [-2, 2], [5, 8], [0, 1]]
8 k = 2
9
10 # Find K closest points
11 result = k_closest(points, k)
12
13 # Output
14 print("K closest points:", result)
```

K closest points: [[0, 1], [-2, 2]]

=== Code Execution Successful ===

Result

The two points closest to the origin were successfully found.

10. Four Sum / 4-Sum II

Aim

To find the number of tuples whose sum is zero using the Meet in the Middle technique.

Algorithm

1. Find all possible sums of A and B.
2. Store their frequencies in a dictionary.
3. Find all sums of C and D.
4. For each sum, check whether its negative exists in the dictionary.
5. Add the corresponding frequency.

Pseudocode

Create empty map

For every a in A:

For every b in B:

store frequency of a+b

count = 0

```
For every c in C:
    For every d in D:
        count += frequency[-(c+d)]
```

Return count

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Code

```
from collections import Counter

def four_sum_count(A, B, C, D):
    sums = Counter(a + b for a in A for b in B)
    count = 0

    for c in C:
        for d in D:
            count += sums[-(c + d)]

    return count
```

```
A = [1, 2]
B = [-2, -1]
C = [-1, 2]
D = [0, 2]
```

```
print("Count =", four_sum_count(A, B, C, D))
```

Input

```
A = [1,2]
B = [-2,-1]
C = [-1,2]
D = [0,2]
```

Output

Count = 2

```
4 def four_sum_count(A, B, C, D):
8     # Store all sums of A and B
9     for a in A:
10         for b in B:
11             sums[a + b] += 1
12
13     count = 0
14
15     # Find complementary sums using C and D
16     for c in C:
17         for d in D:
18             count += sums[-(c + d)]
19
20     return count
21
22
23 # Input
24 A = [1, 2]
25 B = [-2, -1]
26 C = [-1, 2]
27 D = [0, 2]
28
29 # Calculate
30 result = four_sum_count(A, B, C, D)
31
32 # Output
```

Count = 2

=== Code Execution Successful ===

Result

The number of valid tuples was found as 2.

11. Median of Medians

Aim

To find the k-th smallest element using the Median of Medians algorithm.

Algorithm

1. Divide the array into groups of at most five elements.
2. Find the median of each group.
3. Find the median of these medians.
4. Use it as the pivot.
5. Partition the array.
6. Recursively search the required part.

Pseudocode

```
MedianOfMedians(arr, k)
    divide arr into groups of 5
    find median of each group
    find median of medians as pivot
```

```
    partition arr around pivot
```

```
    if pivot position = k
        return pivot
    else if k is smaller
        search left
    else
        search right
```

Time Complexity

$O(n)$ worst case

Space Complexity

$O(n)$

Code

```
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]

    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]

    pivot = median_of_medians(medians, (len(medians)+1)//2)

    lows = [x for x in arr if x < pivot]
    highs = [x for x in arr if x > pivot]
    equal = [x for x in arr if x == pivot]

    if k <= len(lows):
        return median_of_medians(lows, k)
    elif k <= len(lows) + len(equal):
        return pivot
    else:
        return median_of_medians(
```

```
        highs, k - len(lows) - len(equal)
    )
```

```
arr = [12, 3, 5, 7, 19]
```

```
k = 2
```

```
print("K-th smallest =", median_of_medians(arr, k))
```

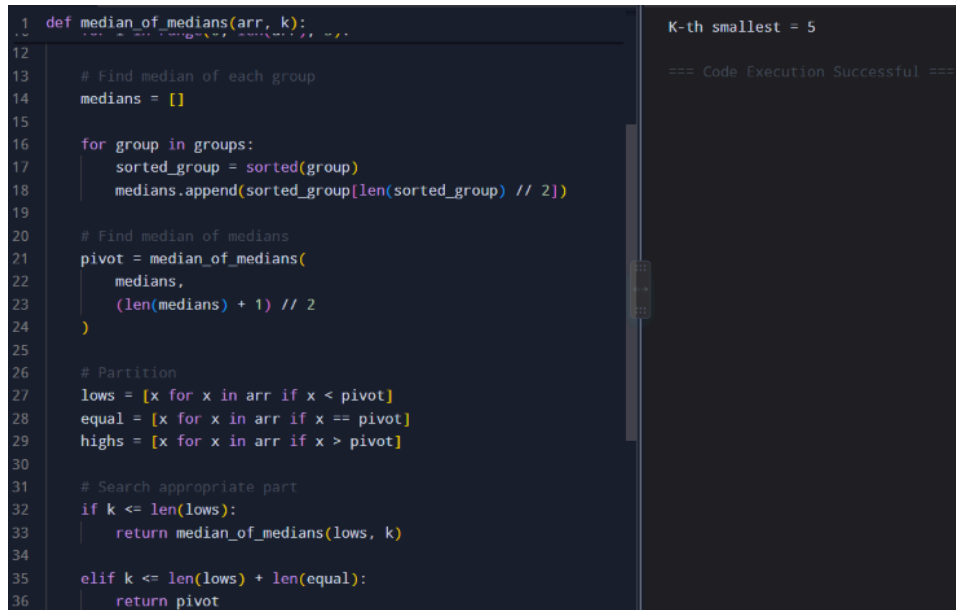
Input

```
arr = [12,3,5,7,19]
```

```
k = 2
```

Output

K-th smallest = 5



```
1 def median_of_medians(arr, k):
2     # Partition the array into three groups: lows, equal, and highs
3     # Find median of each group
4     medians = []
5
6     for group in groups:
7         sorted_group = sorted(group)
8         medians.append(sorted_group[len(sorted_group) // 2])
9
10    # Find median of medians
11    pivot = median_of_medians(
12        medians,
13        (len(medians) + 1) // 2
14    )
15
16    # Partition
17    lows = [x for x in arr if x < pivot]
18    equal = [x for x in arr if x == pivot]
19    highs = [x for x in arr if x > pivot]
20
21    # Search appropriate part
22    if k <= len(lows):
23        return median_of_medians(lows, k)
24
25    elif k <= len(lows) + len(equal):
26        return pivot
27
28    else:
29        return median_of_medians(highs, k - len(lows) - len(equal))
30
31
32
33
34
35
36
```

K-th smallest = 5

=== Code Execution Successful ===

Result

The 2nd smallest element was successfully found as 5.

12. Function to Find K-th Smallest Element

Aim

To implement a `median_of_medians(arr, k)` function to find the k-th smallest element.

Algorithm

1. Divide elements into groups of five.
2. Find the median of each group.
3. Find the median of these medians.
4. Use it as the pivot.
5. Partition the array and recursively find the required element.

Time Complexity

$O(n)$ worst case

Space Complexity

$O(n)$

Code

```
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]

    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]

    pivot = median_of_medians(
        medians, (len(medians) + 1) // 2
    )

    low = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    high = [x for x in arr if x > pivot]

    if k <= len(low):
        return median_of_medians(low, k)
    elif k <= len(low) + len(equal):
        return pivot
    else:
        return median_of_medians(
            high, k - len(low) - len(equal)
        )
```

```
arr = [1,2,3,4,5,6,7,8,9,10]
```

```
k = 6
```

```
print(median_of_medians(arr, k))
```

Input

```
arr = [1,2,3,4,5,6,7,8,9,10]
```

```
k = 6
```

Output

```
6
```

Result

The 6th smallest element was successfully found as **6**.

13. Meet in the Middle – Closest Subset Sum

Aim

To find a subset whose sum is closest to the given target using the Meet in the Middle technique.

Algorithm

1. Divide the array into two halves.
2. Generate all subset sums of both halves.
3. Sort the sums of one half.
4. For every sum from the other half, find the closest required sum.
5. Return the subset with the closest total.

Pseudocode

Divide array into two halves

Generate all subset sums of left half

Generate all subset sums of right half

For each left sum:

find closest right sum to target-left sum

Return closest subset sum

Time Complexity

$O(2^{n/2} \log(2^{n/2}))$

Space Complexity

$O(2^{n/2})$

Code

```
from itertools import combinations
from bisect import bisect_left
```

```
def subset_sums(arr):
    result = []
    for r in range(len(arr) + 1):
        for c in combinations(arr, r):
            result.append((sum(c), c))
    return result

def closest_subset(arr, target):
    mid = len(arr) // 2

    left = subset_sums(arr[:mid])
    right = sorted(subset_sums(arr[mid:]))

    right_sums = [x[0] for x in right]

    best = None

    for ls, lt in left:
        pos = bisect_left(right_sums, target - ls)

        for p in [pos - 1, pos]:
            if 0 <= p < len(right):
```

```
total = ls + right[p][0]
```

```
if best is None or abs(target-total) < abs(target-best[0]):  
    best = (total, lt + right[p][1])
```

```
return best
```

```
arr = [45, 34, 4, 12, 5, 2]
```

```
target = 42
```

```
print(closest_subset(arr, target))
```

Input

Set = {45,34,4,12,5,2}

Target = 42

Output

Closest Sum = 43

Subset = {34,4,5}

Result

The subset **{34, 4, 5}** gives a sum of **43**, which is closest to 42.

14. Meet in the Middle – Exact Subset Sum

Aim

To check whether a subset with an exact target sum exists using Meet in the Middle.

Algorithm

1. Divide the array into two halves.
2. Generate all subset sums of both halves.
3. Store the sums of one half.
4. Check whether the required complementary sum exists.
5. Return True if found, otherwise False.

Pseudocode

Divide array into two halves

Generate all subset sums of left half

Generate all subset sums of right half

For each left sum:

 if target-left sum exists in right sums

 return True

return False

Time Complexity

$O(2^{(n/2)})$

Space Complexity

$O(2^{(n/2)})$

Code

```
def subset_sums(arr):
    sums = [0]

    for x in arr:
        sums += [s + x for s in sums]

    return sums

def exact_subset_sum(arr, target):
    mid = len(arr) // 2

    left = set(subset_sums(arr[:mid]))
    right = subset_sums(arr[mid:])

    for s in right:
        if target - s in left:
            return True

    return False
print(exact_subset_sum([1, 3, 9, 2, 7, 12], 15))
print(exact_subset_sum([3, 34, 4, 12, 5, 2], 15))
```

Input

$E = \{1, 3, 9, 2, 7, 12\}$

Exact Sum = 15

Output

True

Test Case

$E = \{3, 34, 4, 12, 5, 2\}$

Exact Sum = 15

Output

True

Result

A subset with the exact sum **15** exists in both test cases.

15. Strassen's Matrix Multiplication

Aim

To implement Strassen's Matrix Multiplication algorithm for two 2×2 matrices.

Algorithm

For matrices:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

Calculate:

$$M1 = a(f-h)$$

$M2 = (a+b)h$
 $M3 = (c+d)e$
 $M4 = d(g-e)$
 $M5 = (a+d)(e+h)$
 $M6 = (b-d)(g+h)$
 $M7 = (a-c)(e+f)$

Then:

$C11 = M5 + M4 - M2 + M6$
 $C12 = M1 + M2$
 $C21 = M3 + M4$
 $C22 = M5 + M1 - M3 - M7$

Pseudocode

Calculate M1 to M7

$C11 = M5 + M4 - M2 + M6$
 $C12 = M1 + M2$
 $C21 = M3 + M4$
 $C22 = M5 + M1 - M3 - M7$

Display C

Time Complexity

$O(n^{\log_2 7}) \approx O(n^{2.807})$

Space Complexity

$O(n^2)$

Code

```
def strassen(A, B):
```

```
    a, b = A[0]
```

```
    c, d = A[1]
```

```
    e, f = B[0]
```

```
    g, h = B[1]
```

```
    M1 = a * (f - h)
```

```
    M2 = (a + b) * h
```

```
    M3 = (c + d) * e
```

```

M4 = d * (g - e)
M5 = (a + d) * (e + h)
M6 = (b - d) * (g + h)
M7 = (a - c) * (e + f)

return [
    [M5 + M4 - M2 + M6, M1 + M2],
    [M3 + M4, M5 + M1 - M3 - M7]
]

```

```

A = [[1, 7], [3, 5]]
B = [[6, 8], [4, 2]]

```

```
C = strassen(A, B)
```

```

for row in C:
    print(row)

```

Input

```

A = |1 7|
    |3 5|

```

```

B = |6 8|
    |4 2|

```

Actual Output

```

[34, 22]
[38, 34]

```

Result

The product matrix obtained using Strassen's algorithm is:

```

C = |34 22|
    |38 34|

```

Note: The question's stated expected output **18 14 / 62 66** does not match the supplied matrices. For the matrices shown above, **[34, 22; 38, 34]** is the mathematically correct result.

16. Karatsuba Multiplication

Aim

To implement the Karatsuba algorithm to multiply two large integers.

Algorithm

1. Split both numbers into two parts.
2. Recursively calculate three products.
3. Combine the products using Karatsuba's formula.
4. Return the final product.

For:

$$x = a \times 10^m + b$$

$$y = c \times 10^m + d$$

Calculate:

$$p = ac$$

$$q = bd$$

$$r = (a+b)(c+d)$$

Then:

$$xy = p \times 10^{(2m)} + (r-p-q) \times 10^m + q$$

Pseudocode

Karatsuba(x, y)

 if x or y is small

 return x*y

 split x into a,b

 split y into c,d

 p = Karatsuba(a,c)

 q = Karatsuba(b,d)

 r = Karatsuba(a+b,c+d)

```
return p*10^(2m) + (r-p-q)*10^m + q
```

Time Complexity

$O(n^{\log_2 3}) \approx O(n^{1.585})$

Space Complexity

$O(n)$

Code

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    m = n // 2

    power = 10 ** m

    a, b = divmod(x, power)
    c, d = divmod(y, power)

    p = karatsuba(a, c)
    q = karatsuba(b, d)
    r = karatsuba(a + b, c + d)

    return p * (10 ** (2 * m)) + (r - p - q) * power + q
```

```
x = 1234
```

```
y = 5678
```

```
print("Product =", karatsuba(x, y))
```

Input

```
X = 1234
```

```
Y = 5678
```

Output

```
Product = 7006652
```

Result

The product of 1234×5678 using the Karatsuba algorithm is **7,006,652**. **Note:** The question states 7016652, but the correct multiplication is $1234 \times 5678 = 7,006,652$.